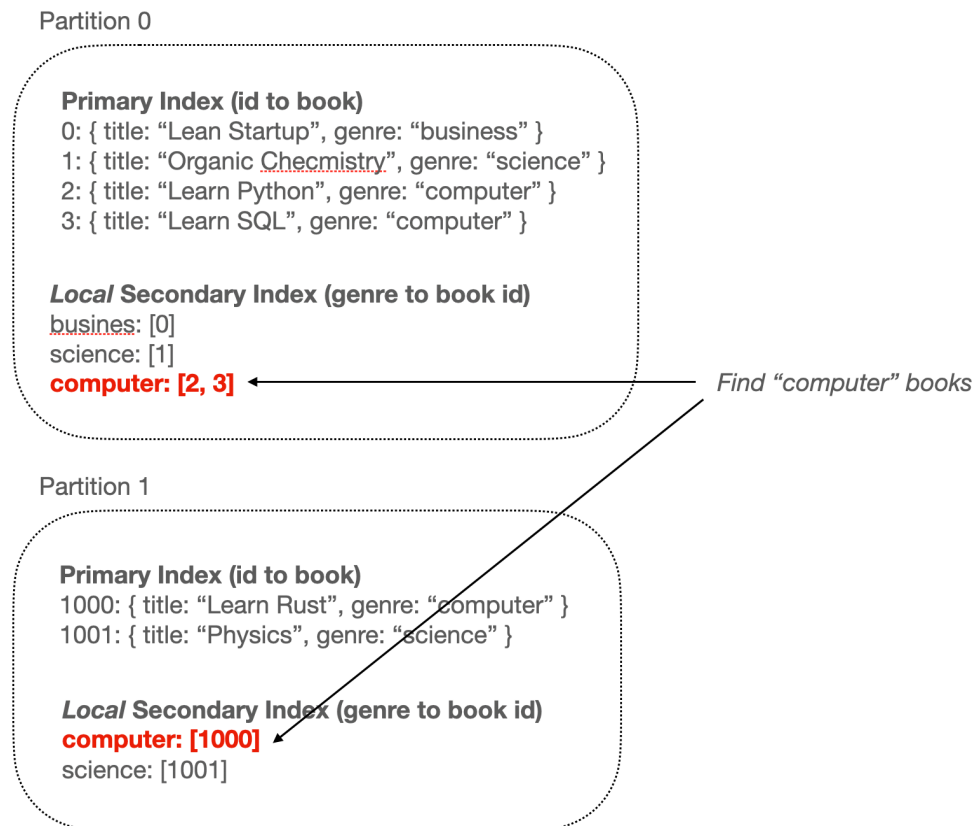


Advanced Database Partitioning Techniques and Key Selection



Secondary indexes

The partition serves as the primary key of your database that identifies a row uniquely. If we only ever need to do primary key lookup, then that's all we need. However, sometimes we do need to look up using other fields. This is where secondary indexes come up.

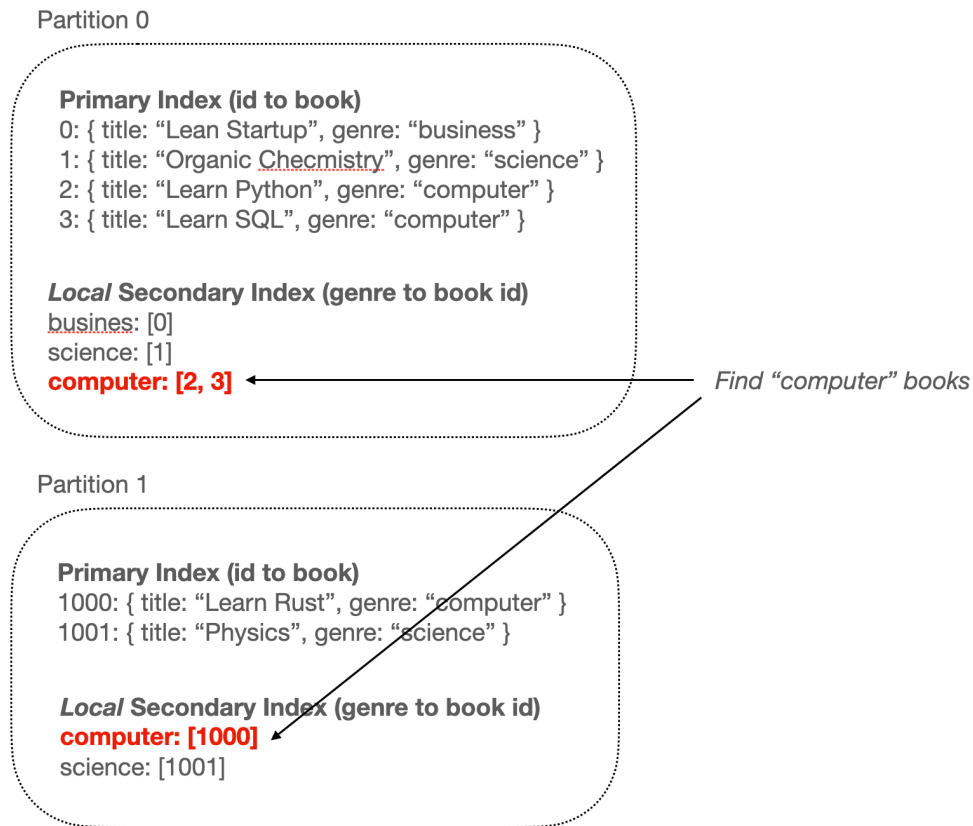
The difficulty with secondary indexes in a partitioned database is that data don't map neatly to partitions. There are two ways to handle this: local secondary index and global secondary index.

Consider a database that stores book info where the partition (primary) key is the book id. And now we want to look up books by their genre quickly so we want to create a secondary index.

Remember, indexes are just dictionary-like extra data stored on disk. The question is where and how to store them.

Local Secondary Index (LSI)

Also called “partition by document”. The idea is for each partition to store its own index. We would store a mapping of genre to book ids in each partition. To find all the books of a genre, the client needs to query all the partitions and aggregate the results. This is called “scatter/gather”.



An example of a local secondary index

Global Secondary Index (GSI)

The idea is to create the index globally, then partition it and put it on each node. The important thing here is that the secondary index partitioning can be different from the primary key index. This is also called “partition by term” because we partitioned the data based on the term we want to create secondary index.

Partition 0

Primary Index (id to book)

0: { title: "Lean Startup", genre: "business" }
1: { title: "Organic Chemistry", genre: "science" }
2: { title: "Learn Python", genre: "computer" }
3: { title: "Learn SQL", genre: "computer" }

Global Secondary Index (genre to book id)

computer: [2, 3, 1000]

Find "computer" books

Partition 1

Primary Index (id to book)

1000: { title: "Learn Rust", genre: "computer" }
1001: { title: "Physics", genre: "science" }

Global Secondary Index (genre to book id)

business: [0]
science: [1]



The advantage of a global secondary index is reads are more efficient: we don't have to query all the partitions with a scatter/gather. The downside is that writing becomes slower because of the additional work we have to do to put the index in the right partition.

There are also other implications of GSI vs LSI. DynamoDB supports only eventual consistency for GSIs, while LSIs offer both eventual and strong consistency. For more information, refer to [DynamoDB's guide on secondary indexes](#).

Rebalancing Partitions

So far, we considered a more or less static scenario. In the real world, however,

- As dataset size increases, we want to add more nodes.
- Nodes may fail, and other machines may need to take over a failed node's data and load.

Rebalancing partitions is necessary to ensure that the data and workload are distributed evenly across all nodes in the new distributed system.

Note that if you use a cloud-based database like the ones we mentioned at the beginning, the scaling and rebalancing are managed by the cloud provider, and you are unlikely to have to worry about it. It is useful to understand how things work behind the scenes nonetheless.

There are several ways to rebalance partitions:

Fixed-size partitioning

The number of partitions is predetermined and remains constant during the system's lifetime. The number of partitions is a few times that of the number of nodes.

Node	Partition
0	0
	4
	8
	12
	16
1	1
	5
	9
	13
	17
2	2
	6
	10
	14
	18
3	3
	7
	11
	15
	19



20 partitions are distributed across 4 nodes

When a new node is added to the system, the system takes a few partitions from other nodes and assigns them to the new node. Similarly, when a node breaks down, the system takes its partitions and distributes them to other nodes. The number of partitions never changes; only

the number of nodes changes. This change of assignment isn't instantaneous, as transferring large amounts of data over the network takes time.

Node	Partition
0	0
	4
	8
	12
	16
1	1
	5
	9
	13
	17
2	2
	6
	10
	14
	18
3	3
	7
	11
	15
	19
4	4
	9
	14
	19



Adding a new node - repartition

The advantage of fixed-size partitioning is it's simple to implement and manage. The downside is the total number of partitions is permanent and it may be difficult to pick the right number of partitions when the dataset size is highly variable.

Dynamic Partitioning

In this approach, the system automatically monitors the workload and data distribution. If it detects an imbalance, it can trigger a partition split or merge to redistribute data across nodes. The number of partitions changes as dataset size changes.

Consistent Hashing

This technique assigns data to nodes based on a consistent hashing algorithm. When a node is added or removed from the system, only a small portion of the keys (data) need to be remapped and moved. This ensures a more balanced data distribution and minimizes the need for data movement during rebalancing.